

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

LAB 01 — INTRODUCTION TO PROGRAMMABLE LOGIC CONTROLLERS;

Practical Lab 01

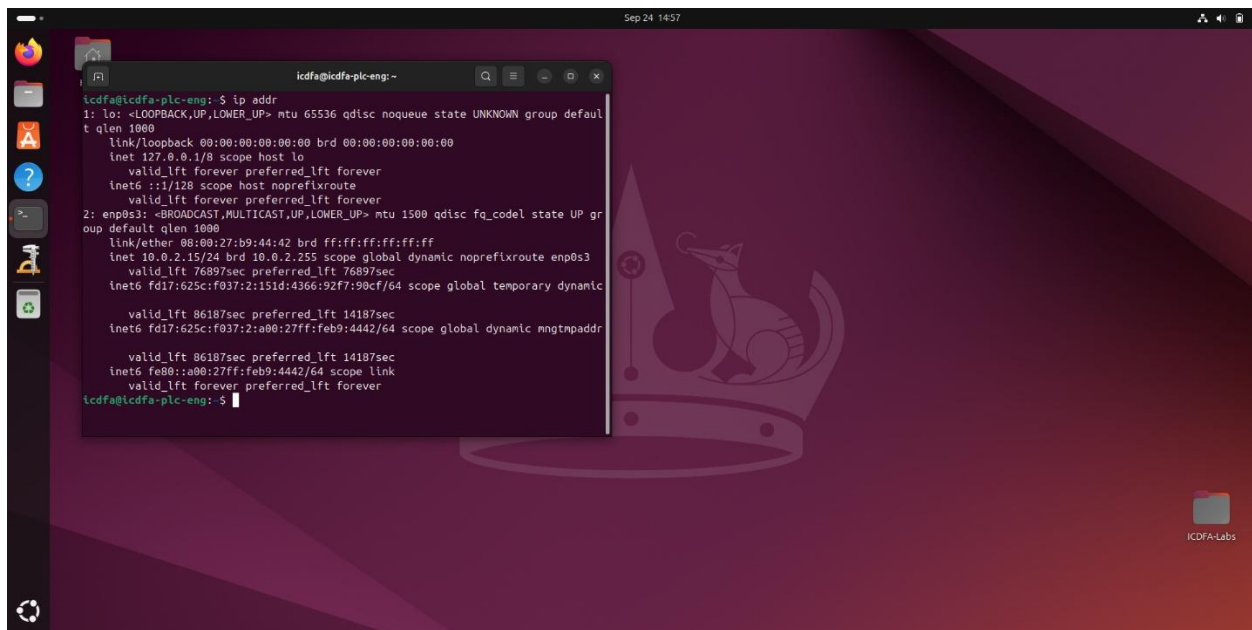
Item	Details
Course	ACICP301 – ICS & SCADA Systems (Practical Laboratories)
Lab	Practical Lab 01: Introduction to Programmable Logic Controllers and OpenPLC
Platform	ICDFA Offline ICS/PLC Virtual Laboratory – OpenPLC VM
Due	02 Oct 2026, 11:59 PM WAT
Practical Pass	50%
Student Name	Nathaniel Temitope Ogunjobi
Registration Number	C11/26/ACIS/17372
Date	21 September 2026

Authorization & Safety: this practical was performed only within the ICDFA-provided PLC virtual machine, imported into the approved virtualization platform and kept isolated from production or unauthorized networks, per the lab access notes. A snapshot of the original VM state was preserved before any changes were made. No production ICS/SCADA system was accessed.

Evidence 1: Laboratory Environment and PLC Identification

The ICDFA-provided PLC virtual machine was imported into the approved virtualization platform (VirtualBox), configured on isolated networking (VirtualBox default NAT, not bridged to any production or organisational network), and started successfully. A snapshot of the clean starting state was preserved before proceeding, so the environment can be restored if needed.

The running VM was confirmed via `ip addr` from a terminal session: `hostname icdfa-plc-eng`, logged in as user `icdfa`, with active interface `enp0s3` assigned address `10.0.2.15/24` (VirtualBox's standard NAT-default range). This differs from the illustrative `192.168.1.10` address referenced in the general lab material, which is expected since the actual exported VM uses VirtualBox's own default NAT networking rather than a fixed address; the important confirmation is that the correct ICDFA-provided VM (by hostname and username) was accessed, not that the IP matches the generic example.



Evidence 1 screenshot

Evidence 2: PLC Architecture Analysis

The seven core architectural components of a PLC, and the function each performs, are summarized below.

Component	Function
Processor / CPU	The “brain” of the PLC. It executes the control program (such as the Motor_Control ladder logic examined in Evidence 4) by continuously scanning inputs, evaluating the logic against them, and updating outputs, cycling through this scan repeatedly for as long as the PLC is running.
Memory	Stores the control program itself (the ladder logic), the current state of every input and output, and any internal variables or timers the logic depends on. Memory is what lets the processor “remember” the latched/running state of an output like MotorOutput between scans, rather than the logic having to be re-derived from scratch every cycle.
Power supply	Converts incoming AC or DC line power into the regulated low-voltage DC power the processor, memory, and I/O modules actually need to operate; a stable power supply is essential since any interruption or fluctuation can reset or corrupt the PLC's operation.

Input module	The interface point where field-sensing devices (push-buttons, limit switches, transmitters) physically connect to the PLC. It converts the real-world electrical signal from each device into a form the processor can read during its input-scan phase.
Output module	The interface point where load devices (motor starters, contactors, valves, indicator lights) physically connect to the PLC. It converts the processor's internal logic result (e.g., MotorOutput energized) into the actual electrical signal needed to drive that physical device.
Rack / chassis	The physical housing that holds the processor, power supply, and I/O modules together, providing the backplane/bus connection that lets these modules communicate with each other and share power from the common supply.
Programming interface / device	The means by which an engineer writes, uploads, monitors, and modifies the PLC's control program, in this lab, the OpenPLC Editor application running on the provided VM, used to open, inspect, and interpret the Motor_Control ladder logic.

Evidence 3: Input/Output Classification

Three input devices and three output/load devices, each identified below, satisfying the minimum of three examples per category:

Device	Input or Output	Discrete/Digital or Analog
Push-button (Start/Stop station)	Input	Discrete/Digital: the button is either pressed or not pressed, a simple on/off state, exactly as used for Start and Stop in this lab's Motor_Control program.
Limit switch (e.g., confirming a valve or gate is fully open/closed)	Input	Discrete/Digital: the switch is either made (contact closed) or not, with no in-between reading.
4–20 mA pressure or temperature transmitter	Input	Analog: reports a continuously variable value across a range (e.g., 0–100 psi mapped to 4–20 mA) rather than a simple on/off state.
Motor starter contactor coil	Output	Discrete/Digital: the coil is either energized (motor running) or de-energized (motor stopped); this is the MotorOutput role in this lab's own ladder logic.
Indicator/pilot light	Output	Discrete/Digital: the lamp is simply on or off, commonly used to show a motor's running or stopped status to an operator.
Variable frequency drive (VFD) speed reference signal	Output	Analog: the PLC sends a continuously variable 0–10V or 4–20 mA signal to set the VFD's exact motor speed, rather than a simple on/off command.

Discrete/Digital I/O vs. Analog I/O

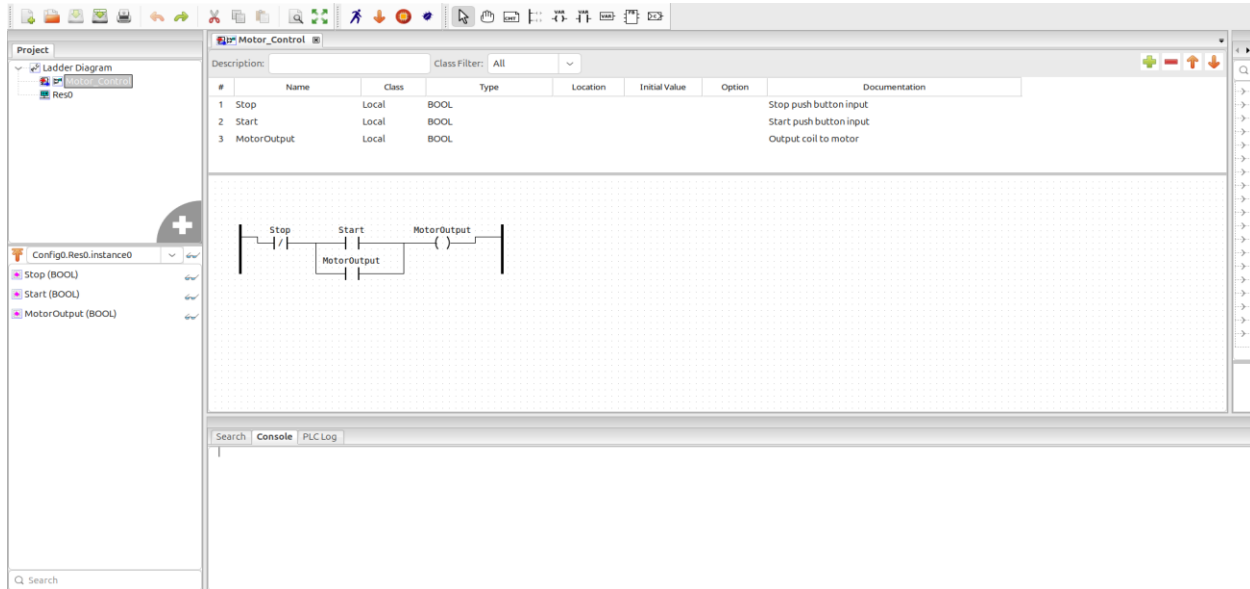
A discrete (digital) I/O signal has only two possible states: on or off, true or false, energized or de-energized. The Start push-button, Stop push-button, and MotorOutput coil in this lab's own Motor_Control program are all discrete I/O: at any instant, each is simply pressed/not pressed or energized/not energized, with no intermediate value possible.

An analog I/O signal, by contrast, represents a continuously variable value across a defined range, such as a 4–20 mA pressure reading or a 0–10V speed reference. The PLC's analog input/output modules convert this continuous electrical value into (or from) a numeric range the processor's logic can compare against thresholds or use directly, which is fundamentally different from the simple true/false evaluation discrete logic performs.

Evidence 4: OpenPLC and Ladder Logic Practical

Procedure Followed

Within the ICDFA PLC VM, OpenPLC Editor was launched from the desktop application menu. Using File > Open, the supplied Ladder Diagram project was opened. Within the project tree, the Motor_Control program was located and opened for inspection, and the ladder logic view was expanded to display the complete program clearly.



Evidence 4 screenshot

Explanation, In My Own Words, of How the Visible Logic Controls the Output

The Motor_Control program shown above is a standard latching (seal-in) circuit, a widely used ladder-logic pattern for starting and holding a motor running from a single momentary button press.

Start contact: a normally open (NO) contact tied to a momentary push-button input. It only allows current/logical continuity while the Start button is actively being pressed.

Stop contact: a normally closed (NC) contact tied to a momentary push-button input. It allows continuity by default and only breaks it while the Stop button is actively pressed, a deliberate fail-safe choice: if the Stop wiring ever fails, the contact fails to the state that stops the motor rather than one that could leave it running unsupervised.

MotorOutput latch contact: a second NO contact, driven by the same MotorOutput coil, wired in parallel with the Start contact. Once the coil energizes, this contact closes and holds ('latches' or 'seals in') the circuit without needing Start to still be pressed.

MotorOutput coil: the output element that actually drives the physical motor output whenever a complete logical path exists to it.

Overall, the coil energizes when the Stop contact is closed (Stop not pressed) AND either the Start contact is closed (Start being pressed) or the latch contact is already closed (motor already running). Pressing Stop breaks both the Start and latch branches at once, since they converge into the same series path before the coil, immediately de-energizing the output. Releasing Start alone does not stop the motor, since the latch branch has already taken over as the current path, this is the entire purpose of the seal-in design: a momentary press starts and holds the motor until a deliberate Stop action (or a fail-safe Stop-line fault) de-energizes it.

Evidence 5: Cybersecurity Implications of Networked PLCs and Industrial Automation

Network-connected PLCs bring genuine operational benefits: a single engineer can monitor and adjust many controllers from a central location instead of visiting each panel physically, faults can be diagnosed remotely, logic and firmware updates can be pushed without a site visit, and process data can be aggregated into a historian for trending. For a facility running a continuous process, this networked visibility reduces downtime and improves response time compared with fully hard-wired relay logic.

The same connectivity that enables this convenience is exactly what introduces cybersecurity risk. A PLC such as the Motor_Control example examined in this lab is designed to trust whatever signal reaches its Start, Stop, and latch inputs; it has no inherent way to verify that a command genuinely originated from an authorized operator rather than an attacker who has gained network access. Once a PLC is reachable over a network, especially one poorly segmented from corporate IT systems, physical access to the panel is no longer required to cause harm: an attacker could potentially issue unauthorized Start/Stop commands, alter logic or timing, or undermine the very fail-safe assumption this lab's Stop-contact design depends on.

Because a PLC directly controls physical equipment, the consequence of a successful attack is not merely data loss but potential equipment damage, safety incidents, or environmental harm. This is why industrial network segmentation, strict access control, and continuous monitoring matter for PLCs at least as much as they do for any IT system, arguably more, given that what a PLC is ultimately connected to is a physical process rather than only data.

Knowledge Check

1. What is the difference between an input module and an output module?

An input module receives signals from field-sensing devices (buttons, switches, sensors) and passes them into the PLC's processor for evaluation; it is how the PLC learns about the current state of the process. An output module sends signals from the PLC's processor out to load devices (motors, contactors, valves, transformers) to actually cause an action in the physical process; it is how the PLC acts on what it has decided based on its logic and the inputs it received.

2. Why is a normally closed contact commonly associated with stop or emergency-stop logic?

A normally closed (NC) contact fails safe: it allows current to flow by default and only breaks that flow when the button is pressed or when the wiring/power to it is lost. Using NC logic for Stop or E-Stop means that if a wire breaks, a connector fails, or the button loses power for any reason, the contact behaves exactly as if Stop had been pressed, de-energizing the output. If a normally open (NO) contact were used instead, the same wiring fault would leave the Stop function silently disabled, since a broken NO circuit simply never closes, meaning the operator could believe Stop is available when it is not. NC logic ensures a fault trends toward the safe (stopped) state rather than the unsafe (running, uncommandable) state.

3. Give one example of a discrete I/O signal and one example of an analog I/O signal.

Discrete I/O example: a limit switch confirming a conveyor gate is fully closed (on/off, binary). Analog I/O example: a 4–20 mA level transmitter reporting the fill level of a storage tank as a continuous value rather than a simple on/off state.

4. What operational risk can arise when industrial automation devices become network-connected?

Networking automation devices like PLCs increases the attack surface available to a malicious actor: a device that once required physical presence at a panel to influence can potentially be reached, monitored, or commanded remotely if the network is not properly segmented and secured. Because a PLC directly drives physical outputs (motors, valves, contactors), unauthorized network access could allow an attacker to issue false commands, disable safety interlocks, or disrupt a process in ways that cause equipment damage, safety incidents, or extended downtime, risks that a purely physically-wired relay system never faced in the same way.

Student Declaration

- I confirm that this submission is my own work and that all sources, screenshots, simulations, and calculations used have been honestly presented.
- I understand that unsafe electrical practice, copied work, or fabricated screenshots may attract disciplinary action.

Student Name	Nathaniel Temitope Ogunjobi
Registration Number	C11/26/ACIS/17372
Signature	
Date	21 September 2026